

CANDIDATO:

FREDDY ANDRES GORDILLO CORDOVA

@FREDIIAN

2026-09-29

Introducción a la propuesta arquitectónica

Antes de definir la arquitectura, los diagramas y la infraestructura tecnológica, es necesario partir de una premisa fundamental: la solución debe responder primero a las necesidades reales del negocio.

Actualmente existen variables que todavía deben ser levantadas o validadas con mayor detalle, como la cantidad esperada de clientes, el volumen transaccional, los sistemas existentes y sus restricciones, las capacidades del equipo de desarrollo y operaciones, los niveles de servicio requeridos, el presupuesto disponible, los requisitos regulatorios y las proyecciones de crecimiento.

Estas variables pueden cambiar de forma importante una decisión arquitectónica. Por esta razón, la propuesta presentada debe considerarse una arquitectura de referencia, construida sobre los requisitos disponibles y preparada para evolucionar a medida que se obtenga mayor información del negocio y de la operación.

El objetivo no debe ser diseñar desde el inicio una plataforma excesivamente compleja pensando en un crecimiento hipotético a muy largo plazo. La arquitectura debe ser proporcional al problema actual, sostenible económicamente y operable por el equipo disponible, pero al mismo tiempo debe permitir crecer sin requerir un rediseño completo cuando aumenten los clientes, las transacciones o los servicios.

Para lograrlo, la arquitectura debe diseñarse de manera evolutiva, modular y orientada al dominio del negocio. En lugar de organizar la solución únicamente alrededor de tecnologías, bases de datos o aplicaciones, es recomendable estructurarla alrededor de capacidades de negocio claramente identificadas, por ejemplo: Clientes, Cuentas, Transferencias, Pagos, Movimientos, Notificaciones, Seguridad, Autenticación, Fraude y Cumplimiento.

Este enfoque puede apoyarse en Domain-Driven Design (DDD) para identificar los dominios y subdominios del negocio, definir límites claros mediante Bounded Contexts y reducir el acoplamiento entre componentes. Cada servicio debe tener una responsabilidad claramente relacionada con una capacidad del negocio y evitar convertirse en un servicio genérico que concentre múltiples responsabilidades.

También es recomendable utilizar prácticas de Business Capability Mapping para relacionar los objetivos estratégicos de la entidad con las capacidades que debe soportar la plataforma. Esto permite que las decisiones tecnológicas puedan justificarse a partir del negocio y no únicamente desde una perspectiva técnica.

Para el levantamiento y gestión de requisitos puede utilizarse BABOK, complementándolo con historias de usuario, casos de uso y criterios de aceptación. Los requisitos no funcionales deben analizarse mediante atributos de calidad como disponibilidad, seguridad, rendimiento, escalabilidad, resiliencia, mantenibilidad, observabilidad y recuperación ante desastres.

Desde el punto de vista arquitectónico, puede utilizarse el modelo C4 para documentar progresivamente la solución mediante diagramas de contexto, contenedores y componentes. Las principales decisiones arquitectónicas deberán registrarse mediante Architecture Decision Records (ADR), documentando el problema, las alternativas evaluadas, la decisión tomada y sus principales consecuencias.

Para el diseño de los servicios se recomienda favorecer principios como alta cohesión, bajo acoplamiento, separación de responsabilidades y autonomía de los servicios**. Cuando exista una capacidad de negocio suficientemente independiente, esta podrá evolucionar como un servicio separado. Sin embargo, no todo componente necesita convertirse en un microservicio; la granularidad debe justificarse de acuerdo con la complejidad del dominio, el volumen de transacciones, la autonomía requerida y la capacidad operativa del equipo.

También pueden aplicarse patrones como API Gateway, eventos de dominio, mensajería asíncrona, Outbox Pattern, Idempotencia, Circuit Breaker, Retry con backoff, Saga y CQRS únicamente cuando exista una necesidad concreta que los justifique. La arquitectura debe evitar incorporar patrones únicamente por tendencia tecnológica.

De forma complementaria, las prácticas de DevOps, CI/CD, Infrastructure as Code, observabilidad y automatización de pruebas deben integrarse desde el diseño para reducir riesgos durante la evolución de la plataforma y facilitar despliegues frecuentes, seguros y repetibles.

El dimensionamiento de infraestructura y las decisiones relacionadas con escalabilidad deberán sustentarse progresivamente mediante métricas reales, pruebas de carga, análisis de capacidad, comportamiento transaccional, crecimiento del negocio, costos de infraestructura y objetivos de nivel de servicio definidos mediante SLA, SLI y SLO.

De esta manera, la arquitectura propuesta busca alcanzar un equilibrio entre las necesidades actuales y la capacidad de crecimiento futuro. El objetivo no es construir desde el primer día la arquitectura más compleja posible, sino una solución segura, resiliente, mantenible y alineada con el negocio, que pueda evolucionar de forma controlada conforme aumenten la demanda y las capacidades de la organización.

Bajo estos principios, los siguientes diagramas presentan la arquitectura propuesta desde diferentes niveles de abstracción, comenzando por el contexto del negocio y avanzando progresivamente hacia los contenedores, componentes, servicios e infraestructura tecnológica.

DIAGRAMA C4 DE NIVEL 1

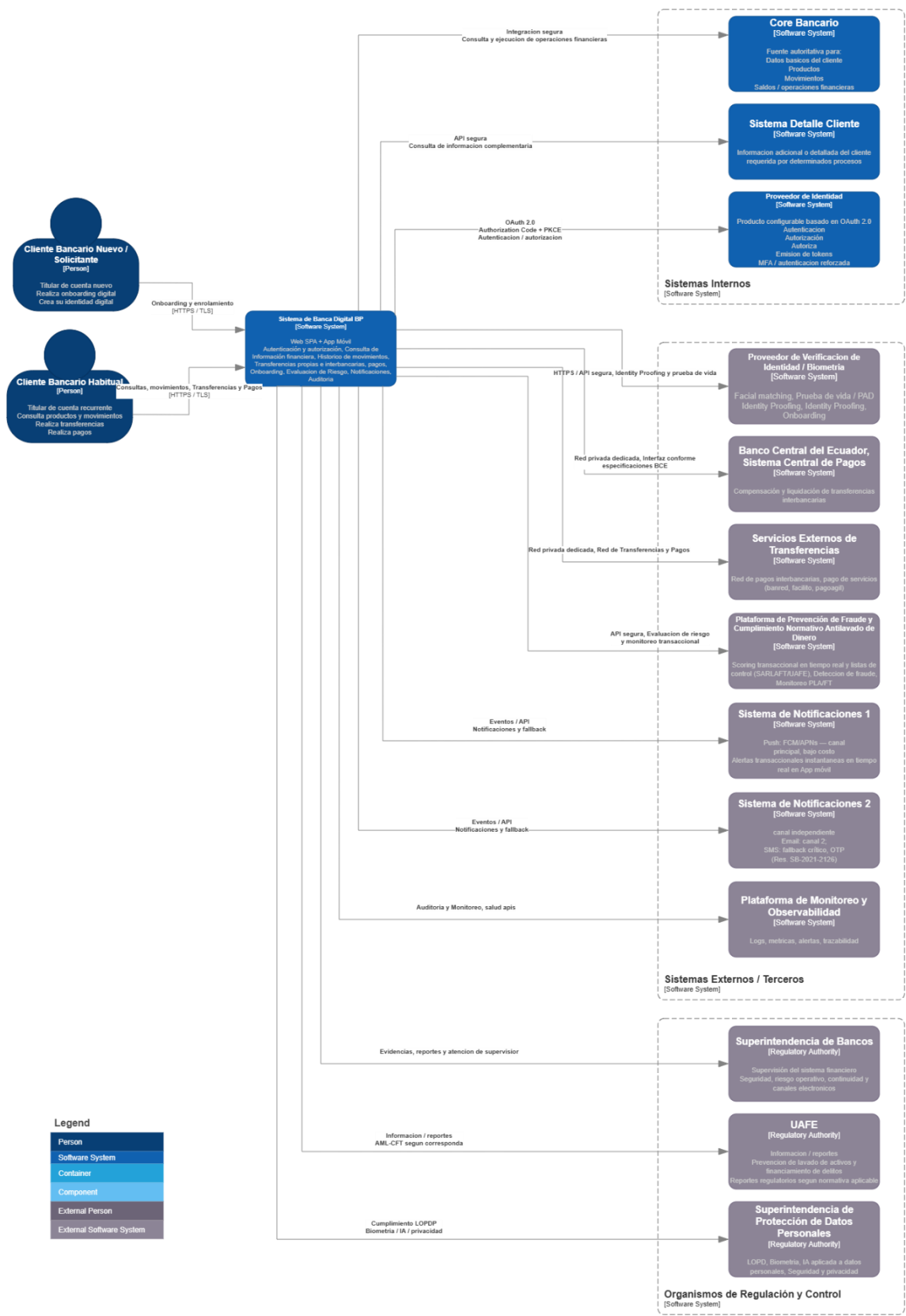


DIAGRAMA C4 DE NIVEL 2

C4 Nivel 2 — Container Diagram · Sistema de Banca Digital BP

Vista técnica de aplicaciones, servicios, datos, mensajería e integraciones. Fondo transparente: API's; arquitectura descrita y ordenada a dominios.

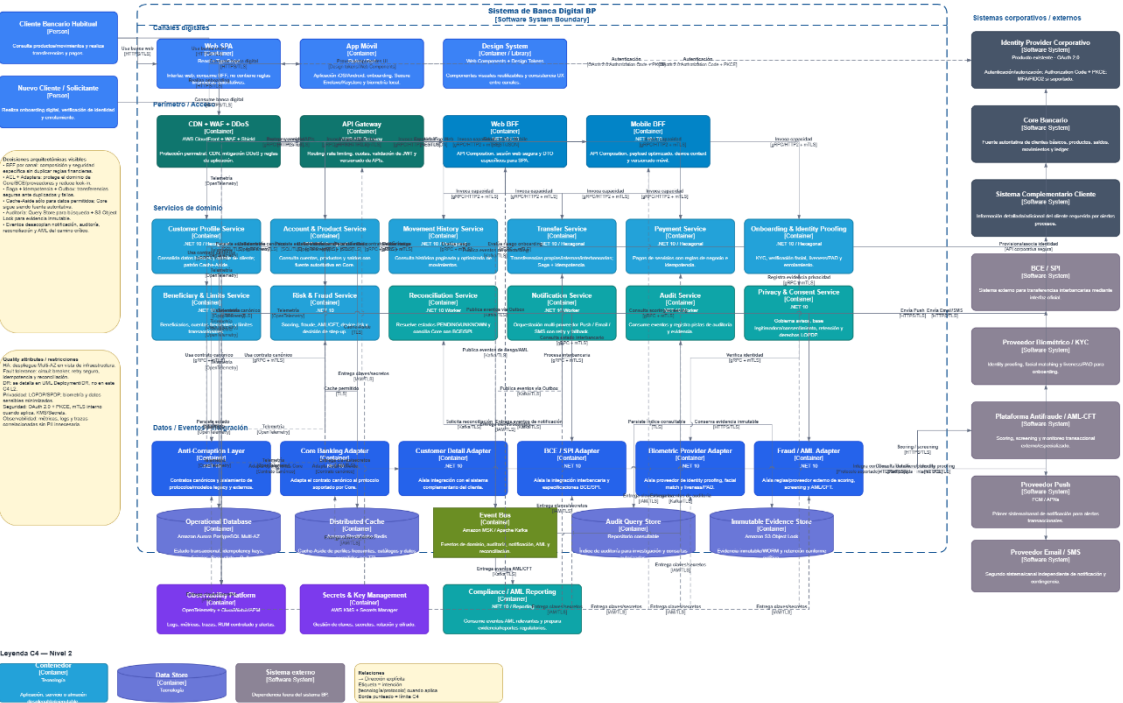
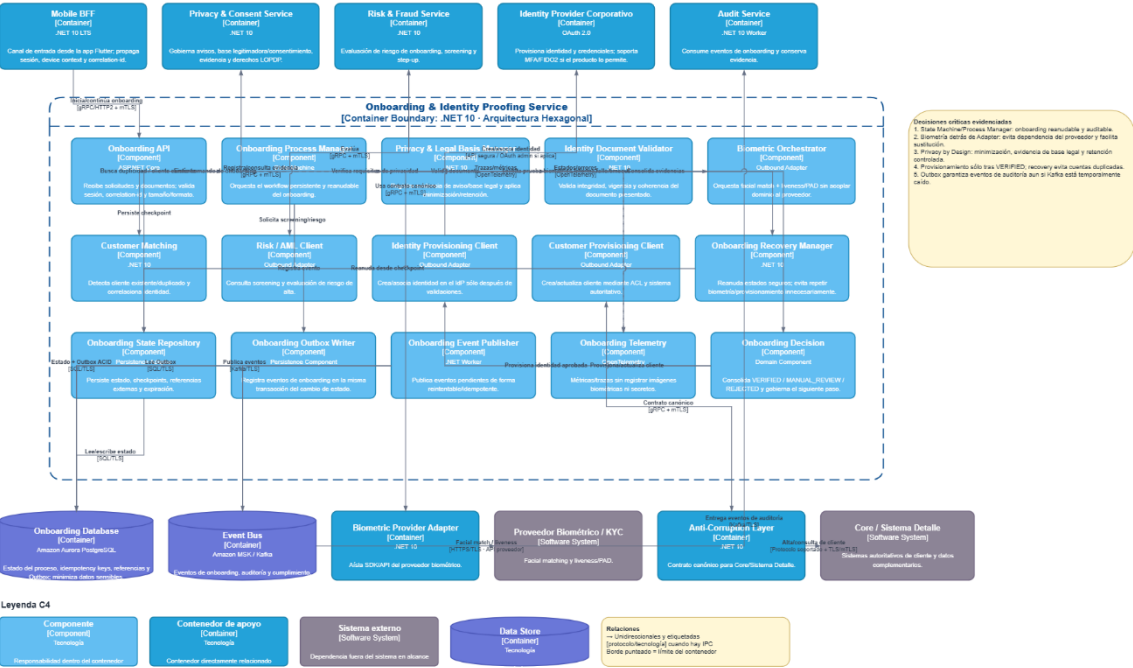


DIAGRAMA C4 DE NIVEL 3

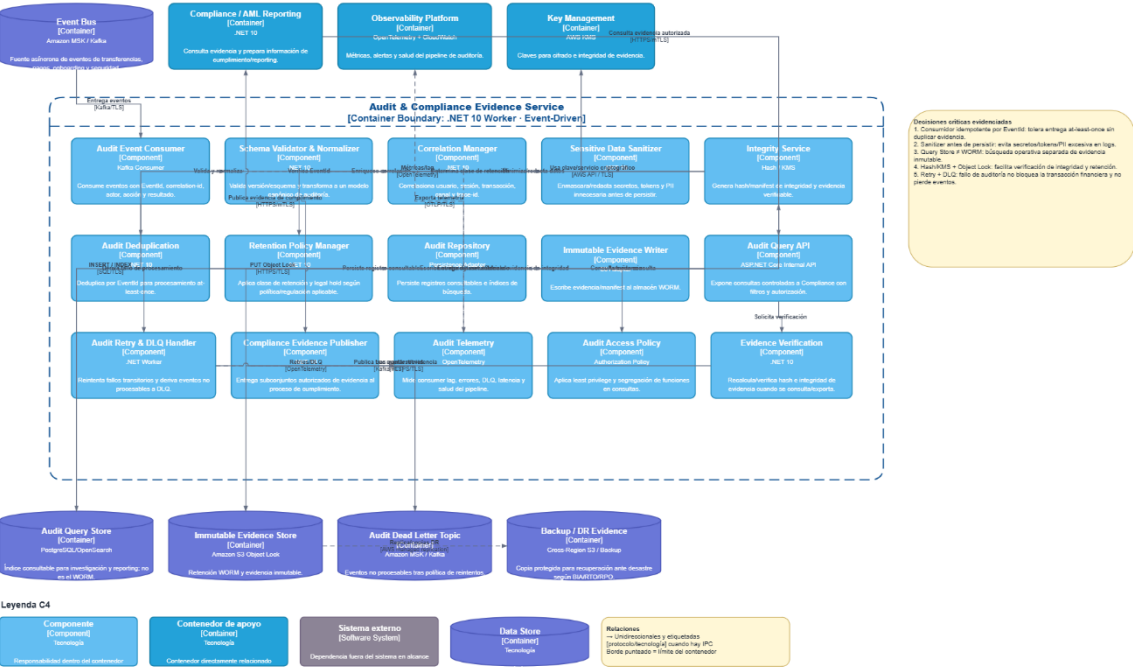
C4 Nivel 3 — Component Diagram · Onboarding & Identity Proofing Service

Scope: un único contenedor · Alta digital, identidad, biometría, privacidad, recuperación e integración KYC



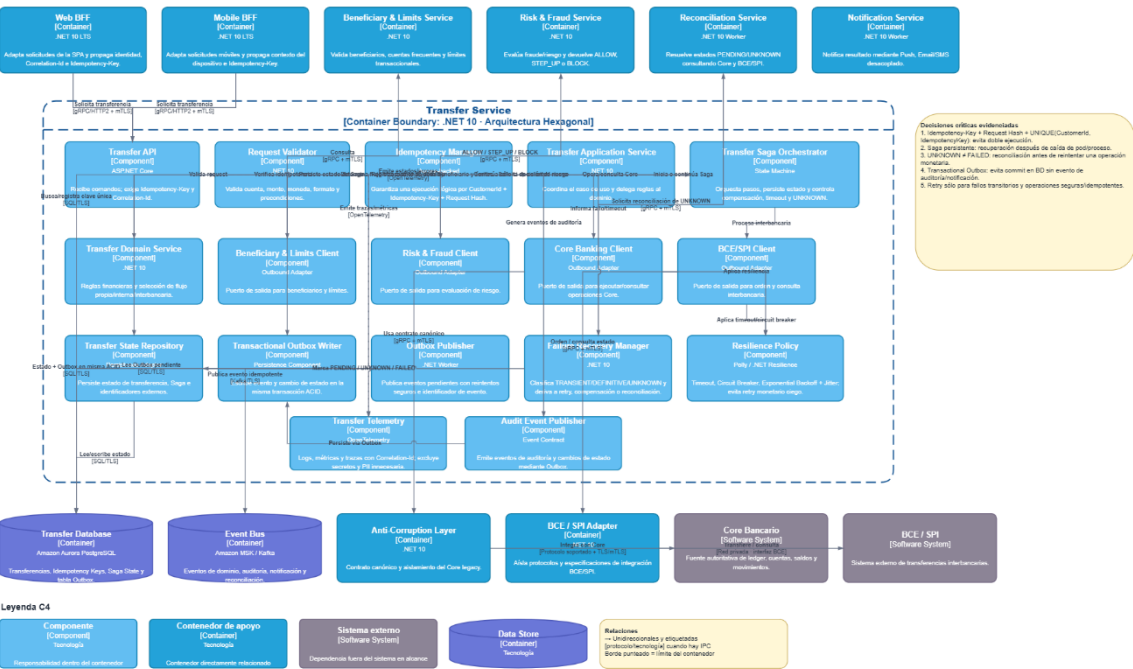
C4 Nivel 3 — Component Diagram · Audit & Compliance Evidence Service

Scope: un único contenedor · Trazabilidad, integridad, minimización de datos y evidencia Inmutable



C4 Nivel 3 — Component Diagram · Transfer Service

Scope: un único contenedor · Componentes responsables de idempotencia, consistencia distribuida y recuperación ante fallos



Infraestructura / Deployment View — Sistema de Banca Digital BP



Decisión	Implementación	Justificación arquitectónica
Alta disponibilidad	EKS + Aurora + Redis + MSK distribuidos Multi-AZ	Evita depender de una única AZ y permite continuidad ante fallo de nodos
Auto-healing	Kubernetes health probes + rescheduling + HPA	Pods fallidos pueden sustituirse automáticamente y capacidad puede escalar
Fault isolation	Subredes privadas + servicios desacoplados + Kafka	Limita propagación de fallos
Core protegido	ACL + red privada + Direct Connect/enlace privado + VPN de contingencia	El Core nunca queda directamente expuesto a Internet

Decisión	Implementación	Justificación arquitectónica
DR	Segunda región en Warm Standby	Balance entre recuperación rápida y costo
Datos transaccionales	Aurora Multi-AZ	Persistencia ACID del estado de la banca digital
Clientes frecuentes	Redis Cache-Aside	Reduce llamadas repetitivas sin convertir el caché en fuente autoritativa
Eventos	MSK/Kafka	Desacopla auditoría, notificaciones, AML y procesos secundarios
Auditoría	Audit DB + S3 Object Lock	Separa consulta operativa de evidencia inmutable
Secretos	Secrets Manager + KMS	Evita secretos en código/imagen/configuración
Observabilidad	OpenTelemetry + CloudWatch/APM	Logs, métricas y trazas correlacionadas
Seguridad de red	WAF + Shield + SG/NACL + egress controlado	Defensa en profundidad
Supply chain	CI/CD + scanning + imágenes firmadas + ECR + IaC	Despliegues repetibles, auditables y automatizados
Conectividad BCE	Red privada dedicada según especificación BCE	Aísla el canal de pagos interbancarios
Proveedores externos	Egress controlado por firewall/NAT	Los microservicios privados no necesitan IP pública